Practical No. 9

Aim

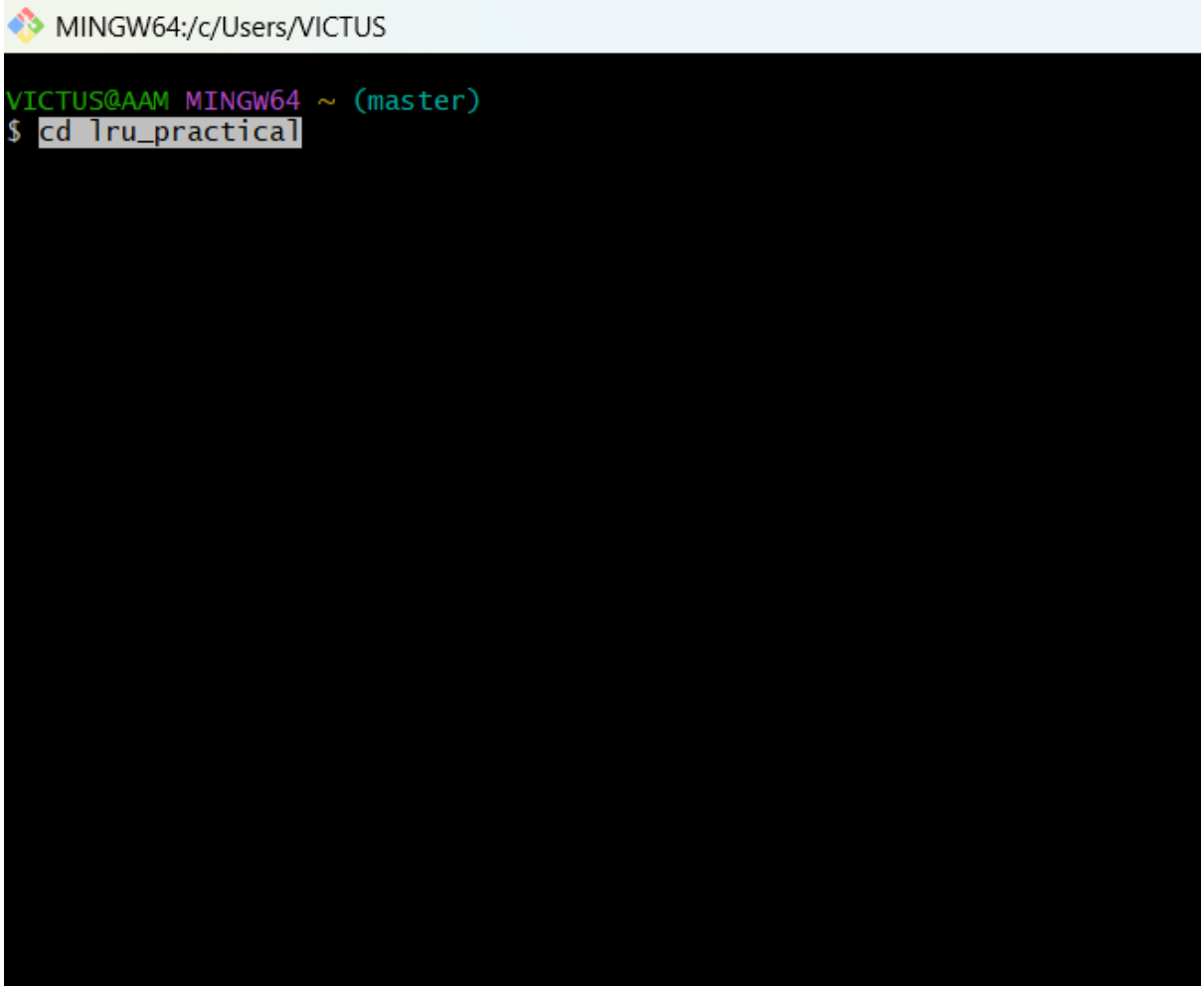
*To write and execute a program to calculate **page hits and page faults using the Optimal Page Replacement algorithm**.*

Step 1: Open Git Bash

Git Bash was opened to execute commands.

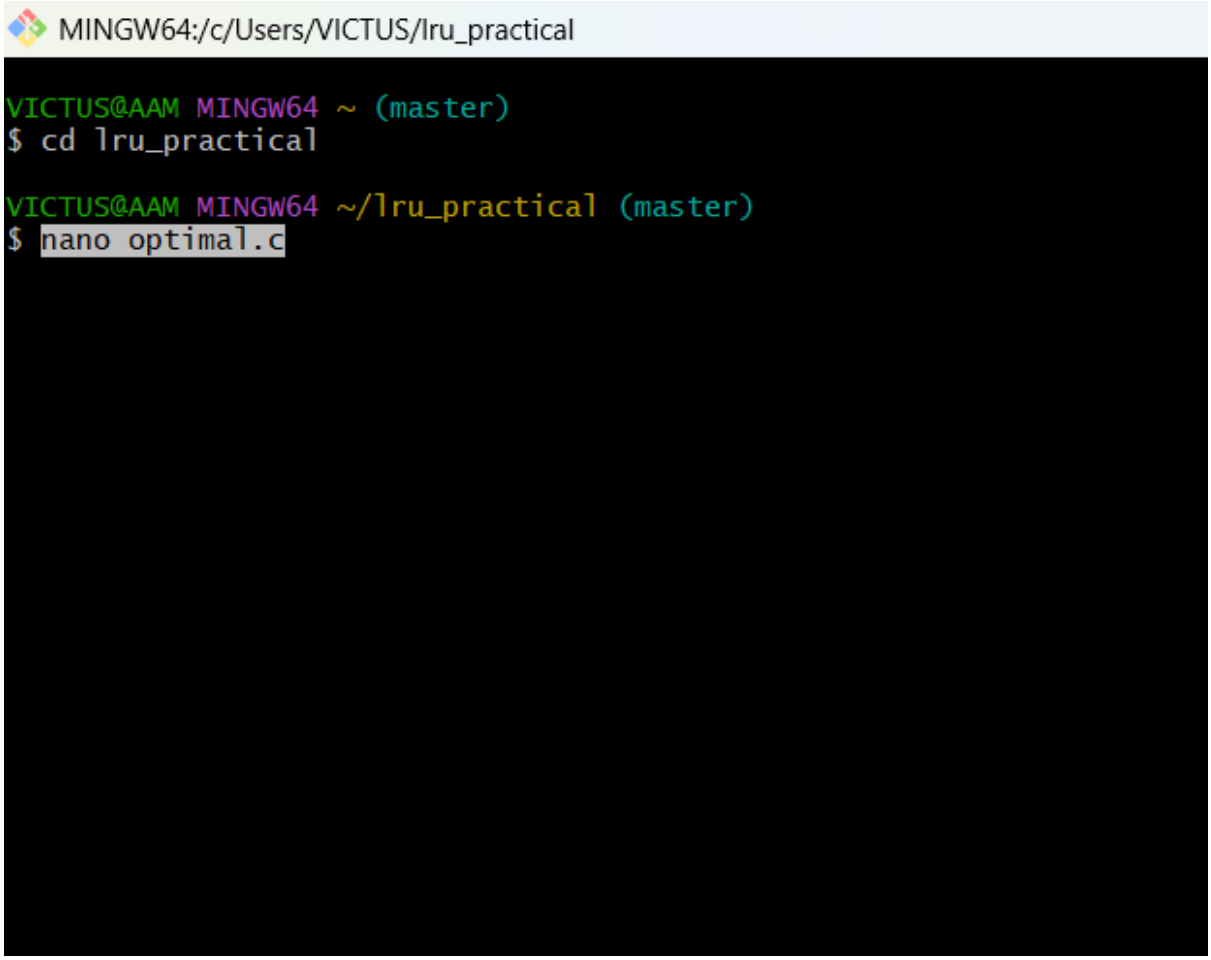
Step 2: Navigate to Directory

The previously created directory was opened:



Step 3: Create C File

A new C file was created using nano editor:



step 4: Write Program

The Optimal Page Replacement program was written in C language with:

- *Number of frames = 3*
- *Page reference string = 1, 2, 3, 4, 1, 2, 5, 1, 2, 3, 4, 5*

The program calculates total page hits and page faults based on future page usage.

MINGW64:/c/Users/VICTUS/lru_practical

GNU nano 8.7

```c
#include <stdio.h>

int main() {
    int frames = 3;
    int pages[] = {1, 2, 3, 4, 1, 2, 5, 1, 2, 3, 4, 5};
    int n = 12;

    int memory[3];
    int i, j, k;
    int hits = 0, faults = 0;

    // initialize
    for(i = 0; i < frames; i++) {
        memory[i] = -1;
    }

    for(i = 0; i < n; i++) {
        int page = pages[i];
        int found = 0;

        // check hit
        for(j = 0; j < frames; j++) {
            if(memory[j] == page) {
                hits++;
                found = 1;
                break;
            }
        }

        // page fault
        if(!found) {
            int index = -1, farthest = i + 1;

            for(j = 0; j < frames; j++) {
                int k;
                for(k = i + 1; k < n; k++) {
                    if(memory[j] == pages[k]) {
                        if(k > farthest) {
                            farthest = k;
                            index = j;
                        }
                        break;
                    }
                }
```

```c
                        farthest = k;
                        index = j;
                    }
                    break;
                }
            }

            // if page not found in future
            if(k == n) {
                index = j;
                break;
            }
        }

        // if all pages will be used soon
        if(index == -1) {
            index = 0;
        }

        memory[index] = page;
        faults++;
    }
}

printf("Total Hits = %d\n", hits);
printf("Total Faults = %d\n", faults);

float hit_ratio = (float)hits / n * 100;
float fault_ratio = (float)faults / n * 100;

printf("Hit Percentage = %.2f%%\n", hit_ratio);
printf("Fault Percentage = %.2f%%\n", fault_ratio);

return 0;
}
```

^G Help ^O Write Out ^F Where Is ^K Cut ^T Execute ^C Locati
^X Exit ^R Read File ^\ Replace ^U Paste ^J Justify ^/ Go To

step 5: *Compile and Execute Program*

The program was compiled and executed using:

MINGW64:/c/Users/VICTUS/lru_practical

```
VICTUS@AAM MINGW64 ~ (master)
$ cd lru_practical

VICTUS@AAM MINGW64 ~/lru_practical (master)
$ nano optimal.c

VICTUS@AAM MINGW64 ~/lru_practical (master)
$ gcc optimal.c -o optimal

VICTUS@AAM MINGW64 ~/lru_practical (master)
$ ./optimal
```

Step 7: Output Observed

The output obtained is:

- *Total Hits = 5*
- *Total Faults = 7*
- *Hit Percentage = 41.67%*
- *Fault Percentage = 58.33%*

MINGW64:/c/Users/VICTUS/lru_practical

```
VICTUS@AAM MINGW64 ~ (master)
$ cd lru_practical

VICTUS@AAM MINGW64 ~/lru_practical (master)
$ nano optimal.c

VICTUS@AAM MINGW64 ~/lru_practical (master)
$ gcc optimal.c -o optimal

VICTUS@AAM MINGW64 ~/lru_practical (master)
$ ./optimal
Total Hits = 5
Total Faults = 7
Hit Percentage = 41.67%
Fault Percentage = 58.33%

VICTUS@AAM MINGW64 ~/lru_practical (master)
$
```

Result

The Optimal Page Replacement program was successfully executed and the page hits and page faults were calculated correctly.